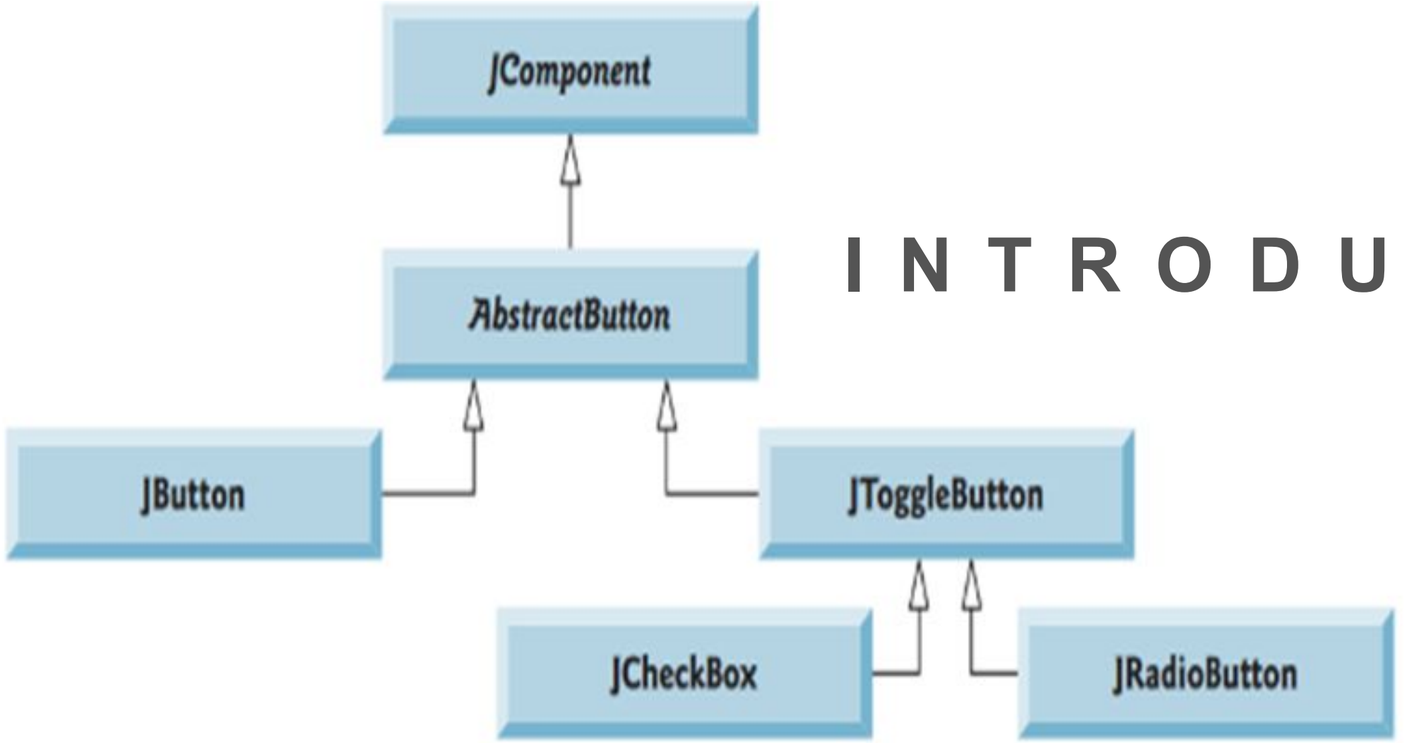
JBUTTON IN JAVA

Tópicos Avanzados de Programación



JESUS JAVIER HERNANDEZ OLVERA

Un **botón** es un componente en el que el usuario hace clic para desencadenar cierta acción. Una aplicación de Java puede utilizar varios tipos de botones, incluyendo **botones de comando**, **casillas de verificación**, **botones interruptores** y **botones de opción**. En la figura 12.14 se muestra la jerarquía de herencia de los botones de Swing que veremos en este capítulo. Como puede ver en el diagrama, todos los tipos de botones son subclases de `AbstractButton` (paquete `javax.swing`), la cual declara las características comunes para los botones de Swing. En esta sección nos concentraremos en los botones que se utilizan comúnmente para iniciar un comando.



INTRODUCCIÓN



Un *botón de comando* (vea la salida de la figura 12.16) genera un evento `ActionEvent` cuando el usuario hace clic en él. Los botones de comando se crean con la clase **`JButton`**. El texto en la cara de un objeto `JButton` se llama **etiqueta del botón**.



Observación de apariencia visual 12.8

El texto en los botones por lo general usa mayúsculas para las primeras letras de las palabras relevantes.



Observación de apariencia visual 12.9

Una GUI puede tener muchos objetos `JButton`, pero cada etiqueta de botón debe ser única en las partes de la GUI en que se muestre. Tener más de un `JButton` con la misma etiqueta hace que los objetos `JButton` sean ambiguos para el usuario.



RESULTADO DE LA EJECUCION

La aplicación de las figuras 12.15 y 12.16 crea dos objetos `JButton` y demuestra que estos objetos tienen soporte para mostrar objetos `Icon`. El manejo de eventos para los botones se lleva a cabo mediante una sola instancia de la *clase interna* `ManejadorBoton` (figura 12.15, líneas 39 a 48).









CODIGO 1 JAVA

```java
package pruebaboton;
import javax.swing.JFrame;
/**
 *
 *
 */
public class PruebaBoton {


    public static void main(String[] args) {
        MarcoBoton marcoBoton = new MarcoBoton();
        marcoBoton.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        marcoBoton.setSize(275,110);
        marcoBoton.setVisible(true);
    }

}
```

CODIGO 2 JAVA P1

```java
package pruebaboton;
import java.awt.FlowLayout;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
import javax.swing.JFrame;
import javax.swing.JButton;
import javax.swing.Icon;
import javax.swing.ImageIcon;
import javax.swing.JOptionPane;

public class MarcoBoton extends JFrame {
    private final JButton botonJButtonSimple;//boton con texto solamente
    private final JButton botonJButtonElegante;//boton con iconos

  public MarcoBoton(){
    super("prueba de botones");
    setLayout(new FlowLayout());

    botonJButtonSimple = new JButton("boton simple");//boton con texto
    add(botonJButtonSimple);//agrega botonJButtonSimple a JFrame
```

```
    Icon insecto1 = new ImageIcon(getClass().getResource("insecto1.gift"));
    Icon insecto2 = new ImageIcon(getClass().getResource("insecto2.gift"));
    botonJButtonElegante = new JButton("boton elegante",insecto1);//establece la imagen
    botonJButtonElegante.setRolloverIcon(insecto2);//establece la imagen de sustitucion

    add(botonJButtonElegante);//agrega botonJButtonElegante a JFrame

    //crea un nuevo manejadorBoton para manejar los eventos de boton
    ManejadorBoton manejador = new ManejadorBoton();
    botonJButtonElegante.addActionListener(manejador);
    botonJButtonSimple.addActionListener(manejador);
}
  private class ManejadorBoton implements ActionListener{
      //maneja evento de boton
      @Override
      public void actionPerformed(ActionEvent evento){
          JOptionPane.showMessageDialog(MarcoBoton.this,String.format("usted oprimio: %s",evento.getActionCommand()));
      }
  }

}
```

CODIGO 2 JAVA P2

El ingeniero en sistemas especializados:



Y como ella me dijo:

Hasta aqui llegamos.

M U C H A S

G R A C I A S

